Instituto Tecnológico y de Estudios Superiores de Monterrey

Campus Puebla

TE3001B: Fundamentación de robótica (Gpo 101)

Profesor: Dr. Rigoberto Cerino Jiménez

Reto semanal 3. Manchester Robotics

Equipo 5

Tania Sofia Arrazola Bello	A01738124
Iñaki Ahedo Madrid	A01738231
Marcos Allen Martínez Cortés	A01737939

02 de marzo de 2026

Resumen

En este reporte se presenta el desarrollo del Reto Semanal 3 propuesto por Manchester Robotics, cuyo objetivo fue implementar el control de velocidad y dirección de un motor DC mediante la integración del framework ROS 2 con un microcontrolador ESP32 utilizando micro-ROS.

El sistema desarrollado permitió establecer comunicación entre una computadora y el microcontrolador mediante el modelo publicación-suscripción de ROS2, donde un nodo envía comandos normalizados que determinan tanto la dirección como la velocidad del motor. Para ello se utilizó modulación por ancho de pulso (PWM) junto con un puente H para el control electrónico del motor.

Durante el desarrollo se implementó un sistema de suscripción en micro-ROS capaz de recibir comandos en tiempo real, procesarlos y generar señales PWM adecuadas. Asimismo, se analizaron fenómenos observados durante la operación, como el cambio de valores en el motor ante señales dentro del mismo ROS DOMAIN ID y la importancia de separar el control de dirección y velocidad.

Los resultados obtenidos demostraron la correcta integración entre hardware y software, validando el funcionamiento del sistema ROS–microcontrolador y permitiendo comprender la interacción entre sistemas embebidos y robótica distribuida.

Objetivos

El objetivo principal es implementar un sistema para controlar una velocidad para un motor DC utilizando micro-ROS, integrando ROS 2 con un microcontrolador ESP32 mediante comunicación por tópicos. Entre los objetivos específicos se encuentran:

- Implementar un nodo micro-ROS suscriptor en ESP32
- Controlar dirección y velocidad de un motor DC mediante un puente H.
- Generar señales PWM utilizando ESP32
- Establecer comunicación entre ROS 2 y hardware físico
- Analizar el comportamiento del motor ante distintas señales de entrada
- Comprender la normalización de señales de control dentro de ROS

Introducción

Robot Operating System (ROS) es un framework de software diseñado para el desarrollo de sistemas robóticos distribuidos, proporcionando herramientas para la comunicación entre procesos mediante nodos, tópicos y mensajes tipados (NVIDIA, s. f.).

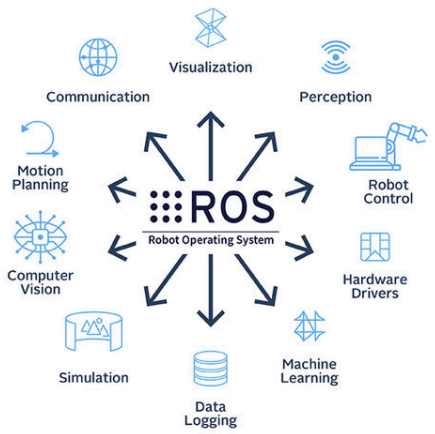

Figura 1: Robot Operating System (ROS).
https://img.directindustry.es/images_di/photo-mg/192516-16646354.jpg

Sin embargo, ROS tradicionalmente opera en computadoras con sistemas operativos completos. Para integrar microcontroladores dentro del ecosistema robótico surge micro-ROS, una extensión de ROS 2 que permite ejecutar nodos en sistemas embebidos con recursos limitados (*Micro-ROS*, s. f.).

Micro-ROS

Micro-ROS adapta las bibliotecas cliente de ROS 2 para microcontroladores, permitiendo que dispositivos como la ESP32 puedan participar directamente en ROS mediante publicación y suscripción de mensajes. Esto habilita arquitecturas distribuidas donde sensores y actuadores forman parte del sistema robótico. En este reto, el ESP32 funciona como un nodo suscriptor que recibe comandos de velocidad desde un agente ROS ejecutándose en una computadora externa (*Micro-ROS*, s. f.).

Sistema de reconexión basado en máquina de estados

El sistema proporcionado por Manchester Robotics utiliza una máquina de estados para asegurar la reconexión automática entre el microcontrolador y el agente micro-ROS. Este mecanismo permite que el sistema recupere la comunicación en caso de pérdida de conexión serial o reinicio del dispositivo, aumentando la robustez del sistema.

PWM en ESP32

El control de velocidad del motor se realiza mediante Pulse Width Modulation (PWM) o modulación por ancho de pulso es una técnica que permite generar una señal analógica a partir de una fuente digital encendiendo y apagando la salida rápidamente. Funciona modificando el ciclo de trabajo (el porcentaje de tiempo que la señal se mantiene en estado "alto" o encendido respecto a su periodo total constante), lo que altera el voltaje promedio entregado a lo largo del tiempo. De esta manera, sistemas digitales que normalmente solo emiten señales de encendido/apagado (como un Arduino) pueden controlar con precisión la cantidad de energía que se envía a una

carga, permitiendo regular la velocidad de un motor, la intensidad de un LED o la posición de un servomotor (Solectro, 2020).

En este reto se utilizó el ESP32 como plataforma de control debido a la compatibilidad directa con micro-ROS. A diferencia de placas como Arduino Uno que utilizan la función `analogWrite()` para generar PWM de forma simplificada, el ESP32 emplea el módulo LEDC (LED Control), un periférico dedicado que permite un control más preciso y configurable de señales PWM. Este módulo requiere seleccionar uno de los 16 canales disponibles, definir la frecuencia de operación (en este caso 980 Hz), establecer la resolución del duty cycle (8 bits en este reto, equivalente a 256 niveles de 0 a 255) y finalmente asociar el canal a un pin GPIO específico. La combinación de frecuencia y resolución determina la granularidad y estabilidad de la señal aplicada al motor; así, el duty cycle controla el voltaje promedio entregado al motor y, por ende, su velocidad de giro, permitiendo un control proporcional y suave dentro del rango operativo definido

DDS

El Data Distribution Service (DDS) es un estándar de middleware de comunicación centrado en datos, definido por el Object Management Group (OMG), que facilita el intercambio de información en tiempo real de manera confiable, escalable y descentralizada bajo una arquitectura de publicación y suscripción.

En ROS 2, DDS constituye la capa subyacente predeterminada responsable del descubrimiento automático de nodos y la transferencia de mensajes; sin embargo, debido a sus altos requerimientos de memoria y procesamiento, su ejecución directa resulta inviable en sistemas embebidos. Para solventar esta limitación, se emplea micro-ROS, un marco de trabajo que implementa el protocolo ligero Micro XRCE-DDS mediante una topología cliente-agente: el microcontrolador (cliente) procesa los datos con una huella de memoria mínima y los transmite a un microprocesador principal (agente), el cual actúa como puente proxy traduciendo bidireccionalmente la información hacia el espacio global de datos DDS, permitiendo así que dispositivos con recursos severamente restringidos se integren de forma nativa y transparente como nodos convencionales dentro de la red de ROS 2. (eProsima, s.f.).

Puente H

Un puente H (H-bridge circuit) es un circuito electrónico diseñado específicamente para controlar tanto la velocidad como la dirección de rotación de motores de corriente continua (DC) utilizando una fuente de alimentación de voltaje fijo. Su nombre se deriva de la disposición topológica de sus cuatro elementos de conmutación (que pueden ser interruptores mecánicos, transistores bipolares, MOSFETs o IGBTs), los cuales forman la letra "H" con el motor situado en la barra central que conecta ambos lados (AspenCore, s.f.).

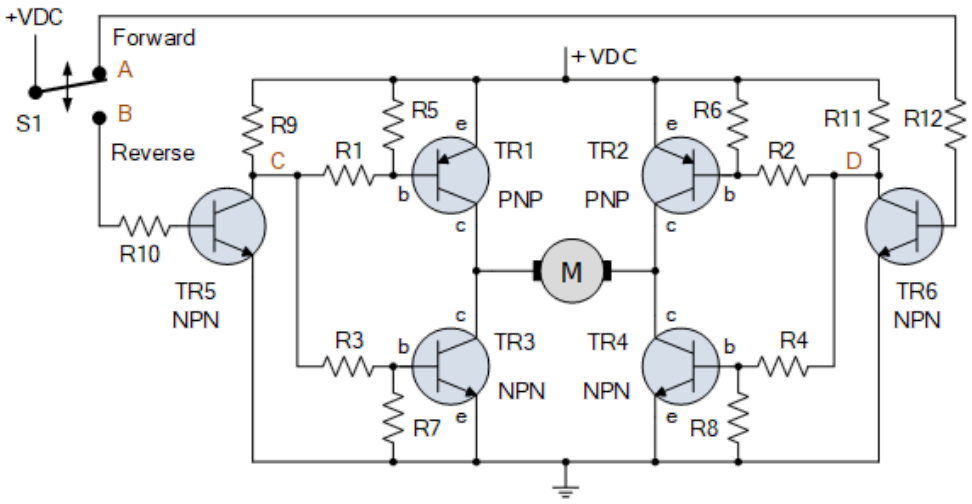

Figura 2: Imagen del puente H en su interior.
https://www.electronics-tutorials.ws/io/h-bridge-circuit.html

Su función principal en el control de motores es permitir la rotación bidireccional. Esto se logra activando (cerrando) los elementos de conmutación en pares diametralmente opuestos. Al encender un par diagonal, la corriente fluye a través del motor en una dirección (por ejemplo, de izquierda a derecha), generando una rotación "hacia adelante". Si se apaga ese par y se enciende el par diagonal opuesto, la corriente fluye en sentido inverso (de derecha a izquierda), invirtiendo así la dirección de giro del motor. Además de cambiar la polaridad para el avance y retroceso, esta configuración permite otras funciones de control dinámico, como el frenado del motor (activando dos interruptores del mismo lado) y el giro libre o inercial (dejando todos los interruptores abiertos), todo esto de forma automatizable mediante lógica digital o modulación por ancho de pulso (PWM) sin necesidad de reconfigurar físicamente las conexiones eléctricas (AspenCore, s.f.).

Duty Cycle

El ciclo de trabajo (duty cycle) es un parámetro fundamental que describe la proporción de tiempo que una señal o sistema se mantiene en su estado activo (encendido o nivel alto) en comparación con el tiempo total de un ciclo completo de operación. Generalmente se expresa como un porcentaje y se calcula mediante la fórmula matemática:

```
Ciclo de trabajo (%) = (Tiempo activo / Periodo total) * 100.
```

Este concepto es la base de la Modulación por Ancho de Pulso (PWM), ya que al variar el ciclo de trabajo de una señal de alta frecuencia, se puede alterar directamente el voltaje o la corriente promedio suministrada a una carga, lo que permite aplicaciones prácticas precisas como el control de velocidad en motores DC, la regulación de intensidad en luces LED y el control de estabilidad en fuentes de alimentación conmutadas.

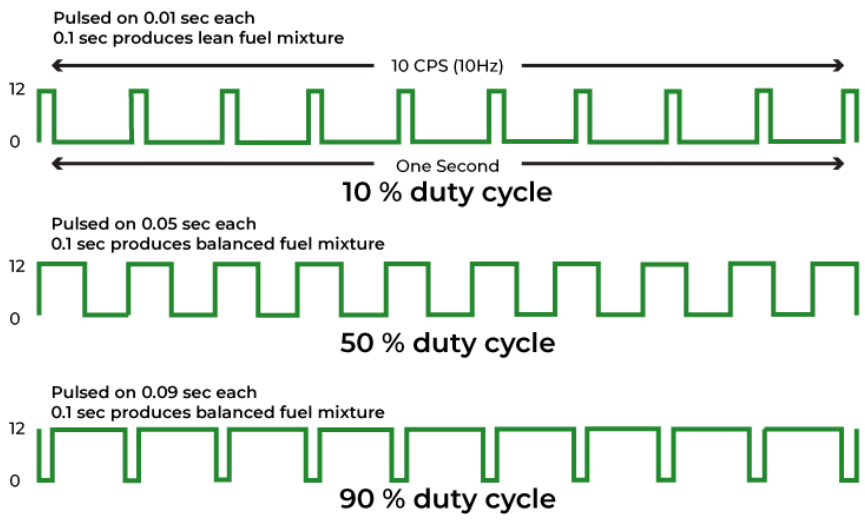

Figura 3: Funcionamiento del Duty Cycle.
https://www.geeksforgeeks.org/electrical-engineering/duty-cycle/

Solución del problema

El Challenge de la semana 3 consistió en desarrollar un nodo micro-ROS capaz de recibir comandos de velocidad desde ROS 2 y traducirlos en señales físicas PWM para controlar bidireccionalmente un motor DC mediante un puente H. La metodología realizada para lograr los objetivos del challenge consistió en las siguientes etapas:

- Configuración del entorno micro-ROS
- Inicialización del nodo y suscripción

A continuación se describe detalladamente cada sección y su pertenencia dentro de la arquitectura.

Arquitectura del sistema

El sistema desarrollado implementa el modelo publicación-suscripción, en el cual la computadora ejecuta el nodo publicador enviando instrucciones al tópico `/cmd_pwm_team`, nombrado de esta manera para evitar conflictos de ROS DOMAIN ID entre diferentes equipos que operen simultáneamente, esto ya que micro_ROS sólo trabaja con el ROS DOMAIN ID 0, mientras que el ESP32 actúa como nodo suscriptor para recibir y procesar dicha señal con el fin de controlar el motor. Esta comunicación es gestionada por el agente micro-ROS, empleando mensajes de tipo `std_msgs/msg/Float32` cuyo valor representa la velocidad del motor normalizada en un rango de -1.0 a 1.0; indicando la máxima velocidad en reversa (-1.0), el motor detenido (0) y la máxima velocidad hacia adelante (1.0). De esta forma, mediante la traducción de los mensajes a señales eléctricas por parte del hardware, la arquitectura logra separar de manera efectiva el nivel de abstracción de software (ROS) del nivel físico (electrónica).

Configuración del entorno micro-ROS

La integración del ESP32 dentro del ecosistema ROS se realizó mediante las siguientes librerías:

```
#include <micro_ros_arduino.h>

#include <rcl/rcl.h>

#include <rclc/rclc.h>

#include <rclc/executor.h>

#include <std_msgs/msg/float32.h>
```

Estas funciones pertenecen al núcleo de micro-ROS y constituyen la base operativa del sistema. Para que estas capacidades se activen, es necesario inicializar primero el transporte entre el ESP32 y el agente micro-ROS mediante la función:

```
set_microros_transports();
```

Esta instrucción establece la comunicación serial directa con el agente que se ejecuta en la computadora. Una vez establecida la vía de comunicación, se procede a inicializar el soporte del sistema a través del siguiente bloque de código, exclusivo de micro-ROS:

```
allocator = rcl_get_default_allocator();

rclc_support_init(&support, 0, NULL, &allocator);
```

En este punto, el sistema configura el contexto de ROS y la asignación de memoria necesaria para los objetos del nodo.

Inicialización del nodo y suscripción

Una vez configurado el entorno, se crea el nodo dentro de ROS con `rclc_node_init_default(&node, "motor", "", &support);` con esta instrucción el ESP32 se registra como un nodo llamado `motor` dentro del sistema distribuido para posteriormente subscribirse al tópico `cmd_pwm_team`. Por el otro lado para que el nodo pueda ejecutar la función correctamente cuando llegue un mensaje es necesario que se inicialice el executor con `rclc_executor_init(&executor, &support.context, 1, &allocator);` donde este es el encargado de monitorear el tópico y ejecutar el callback cada vez que este recibe una información nueva. Finalmente para finalizar el ciclo de operación en micro-ROS l loop del microcontrolador debe asegurarse de que el sistema procese correctamente toda la información de manera continua con:

```
rclc_executor_spin_some(&executor, RCL_MS_TO_NS(100));
```

Esta función es la encargada de gestionar el Executor de micro-ROS, permitiendo que el nodo revise si existen nuevos mensajes en los tópicos suscritos o si hay tareas pendientes. Al definir un tiempo de espera (en este caso, 100 milisegundos), se garantiza que el procesador atienda los callbacks de forma eficiente sin bloquear permanentemente la ejecución de otras tareas locales.

Configuración del hardware

Una vez recibida la señal desde ROS, el ESP32 traduce el valor en señales físicas hacia el puente H. Los pines utilizados fueron:

- `ENA_PIN` es para el control de velocidad (PWM)
- `IN1_PIN` para la dirección 1
- `IN2_PIN` para la dirección 2
- `LED_PIN` para el indicador de estado

El puente H permite invertir la polaridad aplicada al motor y controlar su sentido de giro.

Estrategia de control de velocidad y dirección del motor

Control implementado en el callback

La función que actúa como puente directo entre el ecosistema ROS 2 y el hardware físico es el subscription_callback:

```
void subscription_callback(const void * msgin)
```

En este punto, el sistema traduce el dato abstracto de velocidad (el valor -1.0 a 1.0 recibido de la computadora) en señales eléctricas de PWM (Modulación por Ancho de Pulso) y bits de dirección para los puentes H.

Control de dirección

Como se mencionó antes, la dirección del motor se gestiona de forma independiente al control de velocidad mediante el uso de señales digitales, donde un valor positivo activa el avance, un valor negativo la reversa y un valor cero mantiene el motor detenido.

El valor recibido desde ROS se encuentra normalizado en el rango −1 a 1. Para controlar la velocidad del motor, primero se toma el valor absoluto del comando, ya que la magnitud representa la rapidez de giro, mientras que el signo se utiliza únicamente para la dirección. Posteriormente, este valor se convierte en un `duty cycle` proporcional a la resolución configurada del PWM mediante la siguiente expresión:

```
duty_cycle = (2^resolución - 1) * |valor|
```

Con una resolución de 8 bits, el sistema dispone de 256 niveles discretos de velocidad, es decir, un rango de 0 a 255. En este caso:

- 0 corresponde a motor apagado.
- 255 representa la velocidad máxima.

El valor calculado se envía al módulo LEDC del ESP32 mediante la función `ledcWrite()`, la cual actualiza continuamente el ciclo de trabajo de la señal PWM aplicada al pin de habilitación del puente H. Esto es que al modificar el duty cycle:

- Se cambia el voltaje promedio suministrado al motor.
- Se regula directamente la velocidad de rotación.

De esta manera, ROS trabaja con valores abstractos mientras el microcontrolador realiza la conversión física al rango eléctrico correspondiente.

Por otra parte, la separación entre dirección y PWM evita ambigüedades eléctricas y simplifica el diseño del sistema:

- Dirección: Se controla mediante señales digitales (IN1 e IN2).
- Velocidad: Se regula mediante señal PWM aplicada al pin ENA.

En conjunto, esta estrategia permite un control proporcional y bidireccional del motor DC dentro de una arquitectura modular entre software y hardware.

Resultados

Después de programar el firmware a la ESP32, se debe ejecutar el agente de micro-ros en una terminal con el comando `ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/ttyUSB0` después de haber hecho source en esa terminal. Al correr el comando se debería ver como el agente se ha activado de manera exitosa, sin embargo, en algunos casos puede que después de activarse no se pueda ver que los tópicos hayan sido creados, por lo que se debe de reiniciar el microcontrolador con su respectivo botón de reset. Después de reiniciarlo, se debería ver como se ha creado el tópico el cual en el firmware lo especificamos como suscriptor.

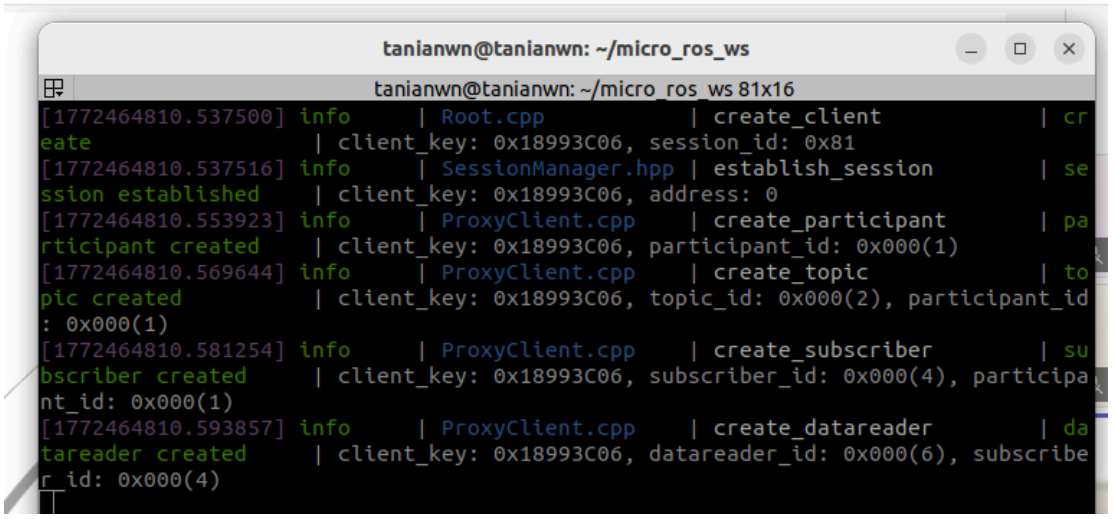

Figura 4: Agente de micro-ROS en ejecución en la terminal.

En otra terminal, se debe asegurar que esté en el mismo `ROS_DOMAIN_ID` que el de la anterior terminal (por lo regular es el 0). Para verificar que el tópico se ha creado exitosamente, se debe ejecutar el comando `ros2 topic list` y verificar que además de los tópicos `/parameter_events` y `/rosout` se debe poder visualizar el tópico `/cmd_pwm` que es al que se suscribe el ESP32. Finalmente, se sabe que el suscriptor está

esperando un valor flotante entre -1.0 a 1.0. Para publicar algún valor entre ese rango, se utiliza el comando `ros2 topic pub /cmd_pwm std_msgs/msg/Float32 "{data: <valor>}"`. Dependiendo si el valor es positivo o negativo, se podrá observar como el motor empieza a girar ya sea a la derecha o a la izquierda con la fuerza especificada tomando en cuenta que -1.0 o 1.0 es el 100% del valor de PWM.

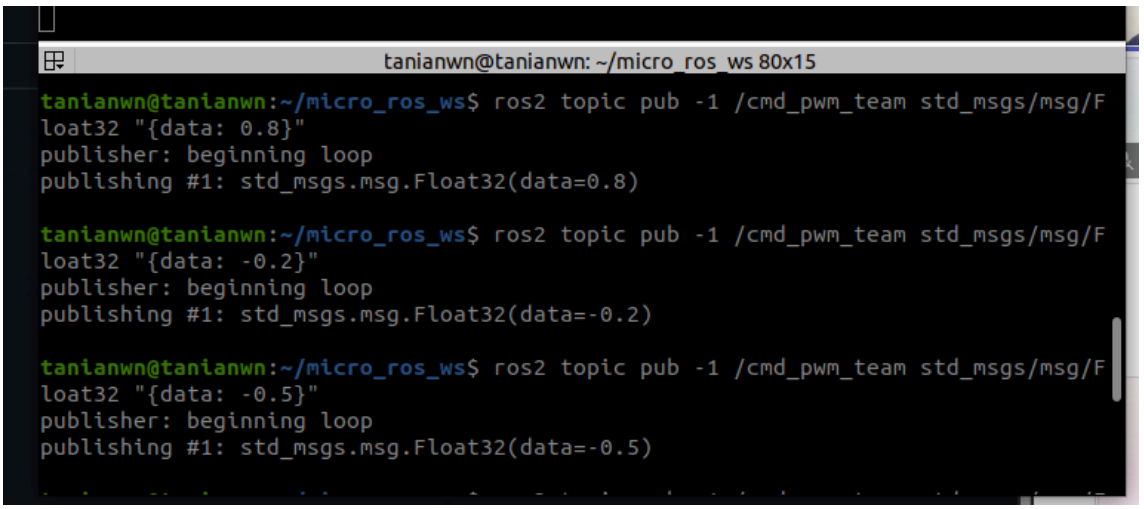

Figura 5: Publicador del tópico `/cmd_pwm` en la terminal.

Preguntas

1. **¿Por qué normalizar entre −1 y 1?**

El microcontrolador toma el valor absoluto para calcular el Duty Cycle (multiplicando por 255 en 8 bits) y usa el signo para conmutar los pines digitales del puente H (IN1/IN2), definiendo así la dirección de giro y la potencia promedio.

2. **¿Por qué separar dirección y PWM?**

Separar la dirección del PWM en un puente H es fundamental para optimizar los recursos del microcontrolador y garantizar la seguridad del hardware, ya que permite que un solo canal de alta frecuencia (PWM) controle la potencia promedio (velocidad) mientras que dos pines digitales simples (GPIO) gestionen la polaridad (giro).

3. **¿Por qué el motor vibra cuando se envían valores pequeños?**

Ocurre principalmente por las limitaciones de la zona muerta (deadband) del motor. Cuando se manda un valor de PWM muy bajo (por ejemplo, un ciclo de trabajo menor al 10% o 15%), el voltaje promedio suministrado no genera el torque suficiente para vencer la inercia y el rozamiento interno de los engranajes. En lugar de girar, el motor recibe pulsos de energía que intentan mover el eje sin éxito, lo que se traduce en vibraciones mecánicas y, a menudo, un zumbido agudo provocado por la frecuencia de conmutación del módulo LEDC de la ESP32.

Conclusiones

Se logró el objetivo del tercer reto semanal de implementar el uso del agente de micro-ROS para controlar el movimiento físico de un motor de corriente directa utilizando un ESP32 cargado con un firmware específico para la creación de un tópico de ROS. Lo más importante fue la creación del nodo `/motor` directamente en el microcontrolador ya que es importante el uso de estos para el control de sensores, motores, etc y no depender de una computadora de mayor consumo de recursos. La publicación de un valor flotante al tópico `/cmd_pwm` que está suscrito el microcontrolador el cual lo traduce a una señal física PWM con dirección y giro, sentó las bases para desarrollar proyectos más complejos que utilice una red de microcontroladores más grande que controle un robot con más capacidades.

La ventaja que puede dar micro-ROS en un ambiente más complejo es de no depender del puerto serial para el intercambio de información ya que permite crear nodos que además de funcionar mediante serial, implementa distintos tipos de comunicaciones como Wi-Fi, TCP, o CAN bus.

Referencias

AspenCore. (s.f.). H-bridge Circuit. Electronics Tutorials. Recuperado el 1 de marzo de 2026, de https://www.electronics-tutorials.ws/io/h-bridge-circuit.html

eProsima. (s.f.). eProsima Micro XRCE-DDS Agent. Micro XRCE-DDS 3.0.1 documentation. Recuperado de https://micro-xrce-dds.docs.eprosima.com/en/latest/agent.html

GeeksforGeeks. (2025, 23 de julio). Duty Cycle. Electrical Engineering. Recuperado el 1 de marzo de 2026, de https://www.geeksforgeeks.org/electrical-engineering/duty-cycle/

Micro-ROS. (s. f.). micro-ROS. https://micro.ros.org/

NVIDIA. (s. f.). *Robot Operating System (ROS)*. Getting Started With Isaac ROS. Recuperado 17 de febrero de 2026, de https://docs.nvidia.com/learning/physical-ai/getting-started-with-isaac-ros/latest/an-introduction-to-ai-based-robot-development-with-isaac-ros/03-what-is-ros.html

Solectro. (2020, 26 de agosto). ¿Qué es PWM y cómo usarlo? Solectroshop. Recuperado de https://solectroshop.com/es/blog/que-es-pwm-y-como-usarlo--n38?srsltid=AfmBOoqQsJc4Nr2W9FVSwRSRqgA0C2RgZwSpfwD1WAJTUVZ9mj25ka6M.